

浙江大学



局域网 P2P 文件传输系统

——基于 BBR 拥塞控制与 AES-256-GCM 加密的安全传输

姓 名 黄鹤翔

学 号 3230106231

院 系 信息与工程学院

指导老师 杜歆

2026 年 6 月

摘 要

本实验设计并实现了一套基于 ECDH 密钥交换与 AES-256-GCM 认证加密的局域网 P2P 文件传输系统。系统采用 UDP 组播进行设备发现, TCP 信令通道协调传输, 多线程并行 TCP 连接结合 BBR 拥塞控制实现高效的文件传输。加密方面, 使用 secp256r1 椭圆曲线 Diffie-Hellman 密钥交换为每对设备协商 256 位共享密钥, AES-256-GCM 以认证加密模式保护所有传输数据。

在跨网络实际测试中, 系统成功实现了两台设备间的安全文件传输。在 RTT 186.8ms 的链路上, 单线程吞吐量达到 67.4 Mbps, 4 线程吞吐量为 63.0 Mbps (500MB 文件)。TEXT_MESSAGE 信令平均延迟 446.6ms。文章详细阐述了 ECDH 密钥协商、AES-GCM 认证加密的数学原理, 以及 BBR 拥塞控制和多线程并行传输的实现机制。

关键词: P2P 文件传输; ECDH 密钥交换; AES-256-GCM; BBR 拥塞控制; 多线程并行传输

目录

摘要	1
一 项目内容	4
1.1 系统架构	4
1.2 协议分层	4
1.3 UDP 组播设备发现	5
1.3.1 组播原理与 IGMP	5
1.3.2 设备发现流程	6
1.3.3 本机 IP 探测 (UDP Connect Trick)	6
1.3.4 组播发送与接收	7
1.3.5 跨子网与 WiFi 隔离注意事项	8
二 算法原理	9
2.1 ECDH 密钥交换算法	9
2.1.1 椭圆曲线数学基础	9
2.1.2 标量乘法	9
2.1.3 ECDH 密钥协商过程	10
2.2 AES-256-GCM 认证加密	10
2.2.1 GCM 模式概述	10
2.2.2 GHASH 多项式哈希	11
2.2.3 GCM 认证标签生成	11
2.2.4 本系统的 GCM 实现方案	11
2.3 BBR 拥塞控制算法	12
2.3.1 BBR 基本原理	12
2.3.2 BBR 状态机	12
2.3.3 与传统算法的比较	13
2.4 多线程并行传输	13
2.4.1 设计动机	13
2.4.2 分块策略	13
2.4.3 并发写磁盘	14

三 算法实现	15
3.1 ECDH 密钥对的生成与交换	15
3.2 ECDH 共享密钥计算	16
3.3 AES-256-GCM 加密与解密	17
3.4 BBR 拥塞控制启用	19
3.5 多线程并行发送实现	19
3.6 数据通道的加密线格式	21
3.7 TCP 信令通信流程	21
3.7.1 短连接模式与状态机	21
3.7.2 信令通信时序	23
3.7.3 非阻塞连接与超时	23
3.8 TCP 文件传输流程	24
3.8.1 传输数据通道概述	24
3.8.2 数据传输时序（多线程模式）	25
3.8.3 Chunk 分片二进制协议	26
3.8.4 接收端多连接并发写入	27
四 项目结果	28
4.1 测试环境	28
4.2 设备发现与信令测试	28
4.3 文件传输性能测试	29
4.3.1 吞吐量对比	29
4.3.2 每线程延迟分解	30
4.4 结果分析	30
4.4.1 单线程已接近带宽上限	30
4.4.2 多线程加速效果分析	30
4.4.3 BBR vs 传统算法对比	31
4.5 加密开销分析	31
4.6 协议开销	31
五 总结	32

一、项目内容

本项目实现了一个完整的局域网 P2P 文件传输工具，支持设备发现、安全通信和高效文件传输。系统采用模块化分层架构，主要包含以下模块：

1.1 系统架构

系统由五个核心模块组成，通过回调机制松耦合协作：

- **设备发现模块** (UDP 组播, 端口 8888)：通过发送和接收 DEVICE_BROADCAST 消息实现局域网设备自动发现，每 3 秒广播一次，10 秒超时判定离线。
- **信令模块** (TCP 短连接, 端口 8889)：负责设备握手 (DEVICE_HELLO) 和传输协调 (FILE_REQUEST / FILE_RESPONSE)。同时承载 ECDH 公钥交换和 TEXT_MESSAGE 聊天功能。
- **传输模块** (TCP 多连接, 端口 8890)：多线程并行文件传输，每个线程独立 TCP 连接，BBR 拥塞控制替代传统滑动窗口协议。支持 zlib 压缩。
- **加密模块**：基于 secp256r1 ECDH 密钥交换 + AES-256-GCM 认证加密，为信令和数据通道提供机密性和完整性保护。
- **Web 界面** (HTTP 端口 8891)：提供设备管理、文件上传/下载、聊天功能的 Web 前端。

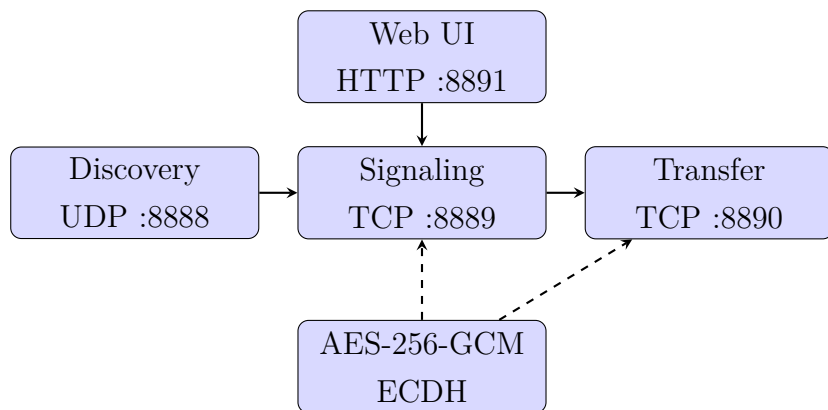


图 1: 系统模块架构与端口分配

1.2 协议分层

系统定义了完整的应用层协议，包含 10 种消息类型：

表 1: 协议消息类型一览

阶段	消息类型	用途
设备发现	DEVICE_BROADCAST	UDP 组播上线广播
	DEVICE_OFFLINE	UDP 组播离线通知
	DEVICE_HELLO	TCP 探活 + ECDH 公钥交换
	DEVICE_HELLO_ACK	应答对方公钥
信令协商	FILE_REQUEST	发起文件传输请求
	FILE_RESPONSE	ACCEPT/REJECT 响应
	TEXT_MESSAGE	聊天文本消息
文件传输	CHUNK_ACK	分片确认
	TRANSFER_DONE	传输完成
	TRANSFER_ERROR	传输错误

1.3 UDP 组播设备发现

1.3.1 组播原理与 IGMP

UDP 组播 (Multicast) 是局域网设备发现的理想机制：一台设备向组播地址发送一个数据包，交换机将该包复制给所有加入该组播组的设备。与广播 (Broadcast) 不同，组播仅在订阅该组的设备上接收，减少不必要的网络负载。

本系统使用 **239.255.255.250:8888** 作为组播地址 (管理范围组播地址 239.0.0.0/8，作用域为本地子网)。设备通过 IGMP (Internet Group Management Protocol) 的 IP_ADD_MEMBERSHIP 系统调用告知内核”加入”该组播组：

```
struct ip_mreq mreq;
mreq.imr_multiaddr.s_addr = inet_addr("239.255.255.250");
mreq.imr_interface.s_addr = inet_addr(local_ip);
setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq));
```

加入组播组后，内核在网络接口的 MAC 层注册对应的组播 MAC 地址 (01:00:5E:7F:FF:FA)，当交换机转发该组播帧时，网卡硬件过滤并递交给 IP 层。

1.3.2 设备发现流程

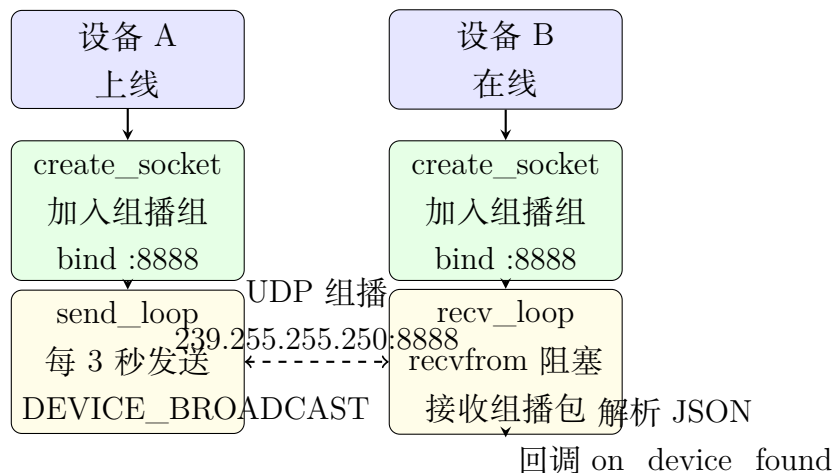


图 2: UDP 组播设备发现流程

核心流程:

1. **创建 UDP 套接字**: `socket(AF_INET, SOCK_DGRAM, 0)`, 设置 `SO_REUSEADDR` 允许多进程绑定同一端口
2. **绑定固定端口**: `bind(sock, 0.0.0.0:8888)`, 监听所有接口
3. **加入组播组**: 对所有非回环 IPv4 接口调用 `IP_ADD_MEMBERSHIP`, 确保多网卡环境中任意接口的组播包均能接收
4. **启动发送线程**: 每 3 秒向组播地址发送 `DEVICE_BROADCAST` JSON 消息, 包含设备 ID、名称、IP、信令端口 (8889)
5. **启动接收线程**: `recvfrom()` 阻塞等待组播数据, 收到后解析 JSON, 过滤自身, 触发 `on_device_found` 回调更新设备管理器
6. **优雅退出**: `stop()` 时先发送 3 次 `DEVICE_OFFLINE` 通知, 再关闭 socket 唤醒阻塞的 `recvfrom()`, `join` 线程

1.3.3 本机 IP 探测 (UDP Connect Trick)

`select_local_ip()` 需要准确获取”本机对外通信的主接口 IP”, 而非回环地址或 Docker 虚拟网桥地址。实现方法:

```

1  std::string DeviceDiscovery::select_local_ip() {
2      // 创建临时 UDP socket, connect 到外网 DNS 地址 (1.1.1.1:53)
3      // connect() 触发内核路由表查询, 确定数据包应从哪个接口发出
4      SOCKET_FD temp_sock = socket(AF_INET, SOCK_DGRAM, 0);
5      struct sockaddr_in remote;
6      remote.sin_family = AF_INET;
7      remote.sin_port = htons(53);
8      inet_pton(AF_INET, "1.1.1.1", &remote.sin_addr);
9      connect(temp_sock, (struct sockaddr*)&remote, sizeof(remote));
10
11     // getsockname() 返回内核为该连接选择的源 IP
12     // 即本机对外通信的主接口地址
13     struct sockaddr_in local_addr;
14     socklen_t len = sizeof(local_addr);
15     getsockname(temp_sock, (struct sockaddr*)&local_addr, &len);
16     // ... 提取 IP 字符串返回 ...
17 }

```

Listing 1: 本机 IP 探测 (device_discovery.cpp)

此方法相比遍历 `getifaddrs()` 的优越之处: 即使存在多个 IP (物理网卡 + Docker 网桥 + VPN), 内核路由表能确定“去往 1.1.1.1 应该走哪个接口”, `getsockname()` 返回的就是该接口的 IP。

1.3.4 组播发送与接收

```

1  void DeviceDiscovery::send_loop() {
2      struct sockaddr_in multicast_addr;
3      multicast_addr.sin_family = AF_INET;
4      multicast_addr.sin_port = htons(DISCOVERY_PORT);
5      inet_pton(AF_INET, "239.255.255.250", &multicast_addr.sin_addr);
6
7      while (m_running.load()) {
8          // 构建包含设备信息的 JSON 广播消息
9          json msg = Protocol::build_device_broadcast(
10              m_device_id, m_device_name, m_local_ip,
11              Defaults::SIGNALING_PORT,           // 注意: 广播的是信令端口
12                                                  (8889)
13              Utils::get_timestamp_ms()           // 非发现端口 (8888)
14          );
15      }
16  }

```



```

13     );
14     std::string msg_str = msg.dump();
15
16     sendto(m_socket, msg_str.data(), msg_str.size(), 0,
17           (struct sockaddr*)&multicast_addr, sizeof(multicast_addr));
18
19     // 分片休眠 (100ms x BROADCAST_INTERVAL*10), 响应停止指令
20     for (uint32_t i = 0; i < Defaults::BROADCAST_INTERVAL * 10; ++i) {
21         std::this_thread::sleep_for(std::chrono::milliseconds(100));
22     }
23 }
24 }

```

Listing 2: 组播发送循环 (send_loop)

关键设计：广播消息中的端口字段填的是 **信令端口 (8889)** 而非组播端口 (8888)，因为接收方将用此端口发起 TCP 连接 (test_connect() 检查设备可达性)。

1.3.5 跨子网与 WiFi 隔离注意事项

1. **组播路由：**Linux 需要主动添加组播路由 `sudo ip route add 224.0.0.0/4 dev <iface>`, 否则组播包被内核静默丢弃
2. **WiFi 客户端隔离：**企业/校园 WiFi 的 AP 通常阻断无线客户端间的组播转发——此时 UDP 发现失效，需依赖 `POST /api/peers/add` 手动添加 IP + TCP DEVICE_HELLO 探活作为后备方案
3. **多网卡兼容：**对每个非回环接口分别调用 `IP_ADD_MEMBERSHIP`，确保组播包从任意接口到达均能接收
4. **TTL：**设置组播 `TTL = 64`，支持跨少量路由器跳数场景

二、算法原理

2.1 ECDH 密钥交换算法

2.1.1 椭圆曲线数学基础

椭圆曲线 Diffie-Hellman (ECDH) 是基于椭圆曲线离散对数问题 (ECDLP) 的密钥协商协议。本系统采用 NIST P-256 曲线 (secp256r1), 定义在有限域 $\mathbb{F}_p$ 上:

$$y^2 = x^3 + ax + b \pmod{p} \quad (1)$$

其中参数:

$$p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$$

$$a = -3$$

$$b = 0x5AC635D8AA3A93E7B3EBBD55769886BC651D06B0CC53B0F63BCE3C3E27D2604B$$

曲线上定义加法群 $\langle G \rangle$, 基点 G 的阶为素数 n 。群运算是**点加法** (几何上: 过两点作直线, 与曲线第三个交点关于 x 轴对称)。

2.1.2 标量乘法

椭圆曲线上的核心运算是**标量乘法**: 给定整数 k 和点 P , 计算 $Q = k \cdot P$ 。

标量乘法采用**倍加算法** (Double-and-Add), 时间复杂度 $O(\log k)$:

Algorithm 1 标量乘法 (Double-and-Add)

```
1: function SCALARMUL( $k, P$ )  
2:    $R \leftarrow \mathcal{O}$  ▷ 无穷远点 (单位元)  
3:   for  $i$  from  $|k| - 1$  down to 0 do  
4:      $R \leftarrow 2 \cdot R$  ▷ 倍点  
5:     if bit  $i$  of  $k$  is 1 then  
6:        $R \leftarrow R + P$  ▷ 加点  
7:     end if  
8:   end for  
9:   return  $R$   
10: end function
```

2.1.3 ECDH 密钥协商过程

两方 Alice (A) 和 Bob (B) 通过以下步骤得到共享密钥:

1. A 生成随机私钥 $a \in [1, n-1]$, 计算公钥 $A_{\text{pub}} = a \cdot G$
2. B 生成随机私钥 $b \in [1, n-1]$, 计算公钥 $B_{\text{pub}} = b \cdot G$
3. A、B 交换公钥 A_{pub} 、 B_{pub}
4. A 计算共享密钥 $S = a \cdot B_{\text{pub}} = a \cdot (b \cdot G) = (ab) \cdot G$
5. B 计算共享密钥 $S = b \cdot A_{\text{pub}} = b \cdot (a \cdot G) = (ab) \cdot G$

正确性由标量乘法的结合律保证:

$$a \cdot B_{\text{pub}} = a \cdot (b \cdot G) = (a \cdot b) \cdot G = (b \cdot a) \cdot G = b \cdot (a \cdot G) = b \cdot A_{\text{pub}} \quad (2)$$

安全性基于 ECDLP: 已知 $A_{\text{pub}} = a \cdot G$ 和 $B_{\text{pub}} = b \cdot G$, 攻击者无法在多项式时间内从 $(G, a \cdot G, b \cdot G)$ 推导出 $(ab) \cdot G$ 。

P-256 曲线的安全强度为 128 位 (与 AES-128 暴力破解难度相当), ECDH 产生的原始共享密钥为 256 位 (32 字节), 直接作为 AES-256 的密钥使用。

2.2 AES-256-GCM 认证加密

2.2.1 GCM 模式概述

Galois/Counter Mode (GCM) 是一种认证加密 (AEAD) 模式, 同时提供**机密性**和**完整性**保护。GCM 将 CTR 模式的加密与 GHASH 多项式哈希相结合, 产生认证标签 (Authentication Tag)。

GCM 的核心组件:

- **CTR 加密**: 对计数器 $J_0, J_1, \dots, J_n$ 分别 AES 加密, 与明文异或产生密文:

$$C_i = P_i \oplus \text{AES}_K(J_i), \quad J_i = \text{inc}(J_0, i) \quad (3)$$

- **GHASH**: 在有限域 $\text{GF}(2^{128})$ 上计算多项式哈希值
- **认证标签**: $\text{Tag} = \text{GHASH}(H, A, C) \oplus \text{AES}_K(J_0)$

2.2.2 GHASH 多项式哈希

首先计算哈希子密钥 H :

$$H = \text{AES}_K(0^{128}) \quad (4)$$

GHASH 对输入数据 $X = (X_1 \parallel X_2 \parallel \cdots \parallel X_m)$ (每块 128 位, 末块填充至 128 位) 在 $\text{GF}(2^{128})$ 上迭代:

$$Y_i = (Y_{i-1} \oplus X_i) \bullet H, \quad Y_0 = 0^{128} \quad (5)$$

其中 $\bullet$ 表示 $\text{GF}(2^{128})$ 上的乘法。该域的定义多项式为:

$$f(x) = x^{128} + x^7 + x^2 + x + 1 \quad (6)$$

GHASH 输入依次为:

1. 附加认证数据 AAD (填充至 128 位对齐) —— 需认证但不加密的数据
2. 密文 C (填充至 128 位对齐)
3. $\text{len}(A)_{64} \parallel \text{len}(C)_{64}$ (长度块, 各 64 位大端)

2.2.3 GCM 认证标签生成

$$\text{Tag} = \text{GHASH}(H, \text{AAD}_{\text{pad}}, C_{\text{pad}}, \text{len}(A), \text{len}(C)) \oplus \text{AES}_K(J_0) \quad (7)$$

其中 $J_0 = \text{IV} \parallel 0^{31}1$ (12 字节 IV 扩展为 16 字节计数器初值)。

解密时: 先验证 Tag 是否匹配——若密文或 AAD 被篡改, GHASH 输出不同, Tag 不匹配, 解密立即失败。这提供了**密文完整性**和**来源认证**双重保护。

2.2.4 本系统的 GCM 实现方案

- **密钥:** ECDH 产生的 32 字节原始共享密钥, 直接作为 AES-256 密钥 (无需 KDF)
- **IV:** 每个消息用 `RAND_bytes()` 随机生成 12 字节, 附在密文前传输
- **Tag:** 16 字节, 附在密文后
- **AAD:** 当前为空 (仅加密数据通过 GHASH)
- **开销:** 每消息额外开销 $12 + 16 = 28$ 字节

2.3 BBR 拥塞控制算法

2.3.1 BBR 基本原理

Bottleneck Bandwidth and Round-trip propagation time (BBR) 是 Google 提出的拥塞控制算法，与传统的基于丢包 (Cubic/Reno) 或基于延迟 (Vegas) 的算法不同，BBR 直接估计链路的两个物理参数：

- **瓶颈带宽** $BtlBw$ ：端到端路径上的最小可用带宽
- **往返传播时延** $RTprop$ ：路径固有的最小 RTT (无排队时)

BBR 的核心思想：**在发送速率接近瓶颈带宽且 inflight 数据量接近 BDP 时，链路达到最优工作点**。其中带宽时延积 BDP 定义为：

$$BDP = BtlBw \times RTprop \quad (8)$$

2.3.2 BBR 状态机

BBR 维护四个状态，周期性轮转：

- **STARTUP** (启动阶段)：类似慢启动，指数增长发送速率，探测 $BtlBw$ 上限
- **DRAIN** (排空阶段)：降低发送速率，排空 STARTUP 阶段积累的队列
- **PROBE_BW** (带宽探测)：以 $pacing_gain$ 循环 $[1.25, 0.75, 1, 1, 1, 1, 1, 1]$ 周期性探测更多带宽
- **PROBE_RTT** (延迟探测)：每 10 秒短暂降低 inflight 到 4 个包，重新测量 $RTprop$

BBR 的 $pacing_rate$ 计算公式：

$$pacing_rate = pacing_gain \times BtlBw \quad (9)$$

发送窗口：

$$cwnd = cwnd_gain \times BDP \quad (10)$$

2.3.3 与传统算法的比较

表 2: BBR vs 传统算法对比

特性	Cubic	Vegas	BBR
拥塞信号	丢包	RTT 变化	带宽/RTT 估计
慢启动	指数增长	指数增长	指数增长（更快）
瓶颈队列	填满	控制	最小化
高 BDP 性能	较差	中等	优秀
对随机丢包敏感	是	否	否

BBR 特别适合本系统的 **P2P 大文件传输** 场景：高 BDP 网络下保持高吞吐的同时避免 bufferbloat，对无线网络随机丢包不敏感。

2.4 多线程并行传输

2.4.1 设计动机

传统单连接 TCP 传输受限于单个拥塞窗口，在高 RTT 链路上带宽利用率低。本系统采用多连接并行传输策略：

$$T_{\text{total}} \approx \frac{S}{\sum_{i=1}^N B_i} \approx \frac{S}{N \cdot B_{\text{avg}}} \quad (11)$$

其中 S 为文件大小， N 为线程数， B_i 为第 i 条连接的吞吐量。

2.4.2 分块策略

文件按 chunk 划分为连续范围：

$$\text{thread}_i : [i \cdot \frac{C}{N} + \min(i, r), (i+1) \cdot \frac{C}{N} + \min(i+1, r)) \quad (12)$$

其中 C 为总 chunk 数， $r = C \bmod N$ 为余数，分配给前 r 个线程各额外 1 个 chunk。

2.4.3 并发写磁盘

接收方每个线程持有独立的 `FileChunkIO` 实例，通过 `pwrite()` 系统调用在不同偏移量写入，利用 POSIX 原子写入保证并发安全：

```
Thread-0: pwrite(fd, data[0], 65536, 0)
Thread-1: pwrite(fd, data[1], 65536, 65536)
Thread-2: pwrite(fd, data[2], 65536, 131072)
```

三、算法实现

3.1 ECDH 密钥对的生成与交换

使用 OpenSSL EVP API 生成 secp256r1 密钥对，密钥以 DER 格式导出后经自定义 Base64 编码在 JSON 消息中传输。

```
1 bool generate_ecdh_keypair(std::string& public_key_b64, std::string&
  private_key_b64) {
2     EVP_PKEY* pkey = nullptr;
3     // 创建 P-256 (secp256r1) 椭圆曲线参数上下文
4     EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new_id(EVP_PKEY_EC, nullptr);
5     if (!ctx) return false;
6
7     bool ok = false;
8     // 初始化密钥生成，设置曲线为 NIST P-256
9     if (EVP_PKEY_keygen_init(ctx) > 0 &&
10         EVP_PKEY_CTX_set_ec_paramgen_curve_nid(ctx, NID_X9_62_prime256v1) >
11         0 &&
12         EVP_PKEY_keygen(ctx, &pkey) > 0) {
13
14         BIO* pub_bio = BIO_new(BIO_s_mem());
15         BIO* priv_bio = BIO_new(BIO_s_mem());
16         // 将公/私钥以 DER 格式写入内存 BIO
17         if (i2d_PUBKEY_bio(pub_bio, pkey) > 0 &&
18             i2d_PrivateKey_bio(priv_bio, pkey) > 0) {
19             // 提取 DER 字节，转为 Base64 字符串
20             char* pub_data = nullptr;
21             long pub_len = BIO_get_mem_data(pub_bio, &pub_data);
22             // 自定义 Base64 编码 (每 3 字节 -> 4 字符)
23             encode((unsigned char*)pub_data, pub_len, public_key_b64);
24             encode((unsigned char*)priv_data, priv_len, private_key_b64);
25             ok = true;
26         }
27     }
28     EVP_PKEY_free(pkey);
29     EVP_PKEY_CTX_free(ctx);
30     return ok;
31 }
```


Listing 3: ECDH 密钥对生成 (utils.cpp)

3.2 ECDH 共享密钥计算

```

1  bool compute_ecdh_shared(const std::string& local_private_b64,
2                          const std::string& remote_public_b64,
3                          std::string& shared_secret) {
4      // Base64 解码私钥和远程公钥为 DER 格式
5      std::string priv_der = b64_decode(local_private_b64);
6      std::string pub_der  = b64_decode(remote_public_b64);
7
8      // 从 DER 加载私钥 (d2i_AutoPrivateKey 自动识别 EC/RSA)
9      const unsigned char* priv_ptr = (const unsigned char*)priv_der.data();
10     EVP_PKEY* priv_key = d2i_AutoPrivateKey(nullptr, &priv_ptr,
11         (long)priv_der.size());
12
13     // 从 DER 加载远程公钥
14     const unsigned char* pub_ptr = (const unsigned char*)pub_der.data();
15     EVP_PKEY* pub_key = d2i_PUBKEY(nullptr, &pub_ptr, (long)pub_der.size());
16
17     // 创建 ECDH 派生上下文
18     EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new(priv_key, nullptr);
19     bool ok = false;
20     // 初始化派生操作, 设置对端公钥
21     if (ctx && EVP_PKEY_derive_init(ctx) > 0 &&
22         EVP_PKEY_derive_set_peer(ctx, pub_key) > 0) {
23         size_t secret_len = 0;
24         // 第一次调用获取所需缓冲区大小
25         if (EVP_PKEY_derive(ctx, nullptr, &secret_len) > 0) {
26             std::vector<unsigned char> secret(secret_len);
27             // 第二次调用获取实际的共享密钥 (32 字节)
28             if (EVP_PKEY_derive(ctx, secret.data(), &secret_len) > 0) {
29                 shared_secret.assign((char*)secret.data(), secret_len);
30                 ok = true;
31             }
32         }
33     }
34 }

```

```

33     EVP_PKEY_CTX_free(ctx);
34     return ok;
35 }

```

Listing 4: ECDH 共享密钥推导 (utils.cpp)

3.3 AES-256-GCM 加密与解密

```

1  bool aes_gcm_encrypt(const std::string& plaintext, const std::string& key,
2      std::string& out) {
3
4      // 随机生成 12 字节 IV (nonce)
5      unsigned char iv[12];
6      if (RAND_bytes(iv, sizeof(iv)) != 1) return false;
7
8      EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
9
10     // 初始化 AES-256-GCM 加密上下文
11     EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), nullptr, nullptr, nullptr);
12     // 设置 IV 长度为 12 字节
13     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, sizeof(iv), nullptr);
14     // 提供密钥和 IV
15     EVP_EncryptInit_ex(ctx, nullptr, nullptr, (unsigned char*)key.data(),
16         iv);
17
18     // 更新明文 -> 密文 (CTR 模式加密)
19     std::vector<unsigned char> ciphertext(plaintext.size() + 16);
20     int len = 0, cipher_len = 0;
21     EVP_EncryptUpdate(ctx, ciphertext.data(), &len,
22         (unsigned char*)plaintext.data(),
23         (int)plaintext.size());
24
25     cipher_len = len;
26     EVP_EncryptFinal_ex(ctx, ciphertext.data() + cipher_len, &len);
27     cipher_len += len;
28
29     // 获取 16 字节 GCM 认证标签 (GHASH 结果 XOR AES_K(J0))
30     unsigned char tag[16];

```

```

28     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, sizeof(tag), tag);
29
30     // 输出格式: [12B IV] [密文] [16B Tag]
31     out.assign((char*)iv, sizeof(iv));
32     out.append((char*)ciphertext.data(), cipher_len);
33     out.append((char*)tag, sizeof(tag));
34     EVP_CIPHER_CTX_free(ctx);
35     return true;
36 }

```

Listing 5: AES-256-GCM 加密 (utils.cpp)

```

1  bool aes_gcm_decrypt(const std::string& ciphertext_with_iv_tag,
2                      const std::string& key, std::string& out) {
3      if (ciphertext_with_iv_tag.size() < 28 || key.size() != 32) return
4          false;
5
6      // 拆分输入: [12B IV] [密文] [16B Tag]
7      const unsigned char* iv      = (unsigned
8          char*)ciphertext_with_iv_tag.data();
9      size_t cipher_len = ciphertext_with_iv_tag.size() - 28;
10     const unsigned char* cipher = iv + 12;
11     const unsigned char* tag     = iv + 12 + cipher_len;
12
13     EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
14     // 初始化解密上下文 (与加密对称)
15     EVP_DecryptInit_ex(ctx, EVP_aes_256_gcm(), nullptr, nullptr, nullptr);
16     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, 12, nullptr);
17     EVP_DecryptInit_ex(ctx, nullptr, nullptr, (unsigned char*)key.data(),
18         iv);
19
20     // 解密
21     std::vector<unsigned char> plaintext(cipher_len);
22     int len = 0, plain_len = 0;
23     EVP_DecryptUpdate(ctx, plaintext.data(), &len, cipher, (int)cipher_len);
24     plain_len = len;
25
26     // 设置期望的 GCM Tag, 调用 Final 进行 Tag 验证
27     // 若 Tag 不匹配, EVP_DecryptFinal_ex 返回 0, 解密失败

```

```

25     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_TAG, 16, (void*)tag);
26     if (EVP_DecryptFinal_ex(ctx, plaintext.data() + plain_len, &len) <= 0) {
27         EVP_CIPHER_CTX_free(ctx);
28         return false; // Tag 验证失败 -> 数据被篡改或密钥错误
29     }
30     plain_len += len;
31     out.assign((char*)plaintext.data(), plain_len);
32     EVP_CIPHER_CTX_free(ctx);
33     return true;
34 }

```

Listing 6: AES-256-GCM 解密与 Tag 验证 (utils.cpp)

3.4 BBR 拥塞控制启用

```

1 bool enable_bbr(SOCKET_FD sock) {
2     #ifdef __linux__
3         // Linux 4.9+ 内核支持 BBR, 通过 TCP_CONGESTION 套接字选项设置
4         const char* cc = "bbr";
5         if (setsockopt(sock, IPPROTO_TCP, TCP_CONGESTION, cc, 3) == 0)
6             return true;
7     #endif
8     // 不支持时退回默认算法 (cubic/reno)
9     return false;
10 }

```

Listing 7: BBR 拥塞控制设置 (platform.cpp)

3.5 多线程并行发送实现

```

1 bool TransferSender::send_file(..., uint32_t num_threads, ...) {
2     // ... 压缩、计算 meta ...
3
4     // 分配 chunk 范围给各线程
5     std::vector<std::thread> workers;
6     std::atomic<uint64_t> global_bytes_sent(0);
7     std::atomic<uint32_t> global_chunks_sent(0);
8

```

```

9   for (uint32_t t = 0; t < num_threads; ++t) {
10      // 均匀分块, 余数分配给前 rem 个线程
11      uint32_t chunks_per = meta.total_chunks / num_threads;
12      uint32_t rem = meta.total_chunks % num_threads;
13      uint32_t start = t * chunks_per + std::min(t, rem);
14      uint32_t end    = start + chunks_per + (t < rem ? 1 : 0);
15
16      workers.emplace_back([..., start, end, t, ...]() {
17          // 每个线程独立连接, 独立发送自己的 chunk 范围
18          send_chunk_range(target_ip, target_port, send_path, meta,
19                          start, end, t, num_threads,
20                          global_bytes_sent, global_chunks_sent,
21                          thread_metrics[t]);
22      });
23  }
24
25  // 等待所有线程完成
26  for (auto& w : workers) w.join();
27  // ... 汇总指标 ...
28 }

```

Listing 8: 多线程并行发送主流程 (transfer_sender.cpp)

```

1  bool send_chunk_range(..., uint32_t start_chunk, uint32_t end_chunk, ...) {
2      SOCKET_FD sock = socket(AF_INET, SOCK_STREAM, 0);
3      setsockopt(sock, SOL_SOCKET, SO_SNDBUF, ...); // 256KB 发送缓冲区
4      NetworkUtils::enable_bbr(sock);              // 启用 BBR 拥塞控制
5      connect(sock, ...);
6
7      // 发送范围头 (FILE_RANGE JSON): 包含 conn_index, total_connections,
8      // chunk 范围
9      send_range_header(sock, meta, start_chunk, end_chunk,
10                       conn_index, total_connections, target_ip);
11
12      FileChunkIO io(file_path, Defaults::CHUNK_SIZE, true);
13      // 顺序发送自己负责的所有 chunk, BBR 自动控制发送速率
14      for (uint32_t i = start_chunk; i < end_chunk; ++i) {
15          Chunk chunk;
16          io.read_chunk(i, meta.file_id, meta.total_chunks, chunk);

```

```

16     send_chunk(sock, chunk); // 加密或明文发送
17 }
18
19 // 发送完成通知并等待接收方 ACK
20 send_range_done(sock, ...);
21 recv_ack(sock, ...);
22 close(sock);
23 }

```

Listing 9: 单线程发送范围 (transfer_sender.cpp)

3.6 数据通道的加密线格式

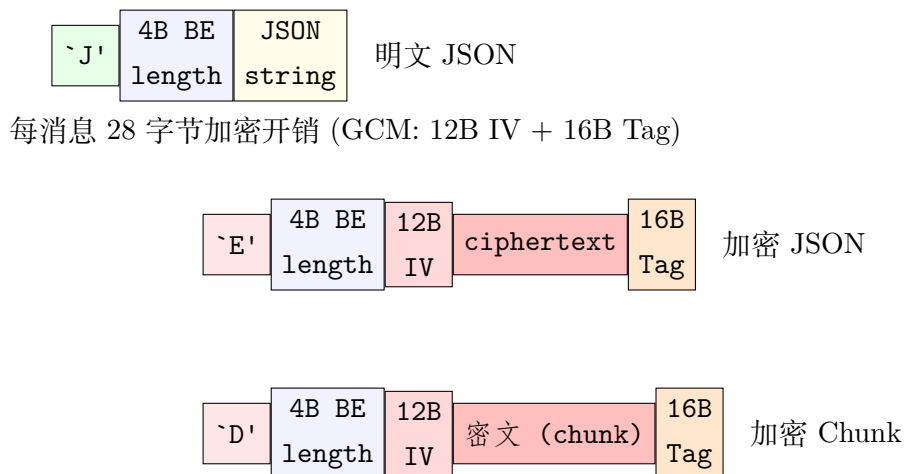


图 3: 数据通道加密线格式

3.7 TCP 信令通信流程

3.7.1 短连接模式与状态机

信令通道（端口 8889）采用 **TCP 短连接** 模式：客户端发起连接 → 发送请求 → 接收响应 → 断开连接。服务端对每个连接创建独立线程处理，处理完毕即关闭连接。

服务端的消息分发状态机如下：

Algorithm 2 SignalingServer 消息分发

```
1: Input: client_sock, client_ip
2: setsockopt(client_sock, SO_RCVTIMEO, 5s)
3: if  $\neg$  recv_json_message(client_sock, msg, client_ip) then
4:   close(client_sock); return
5: end if
6: type  $\leftarrow$  msg["type"]
7: if type = FILE_REQUEST then
8:   创建 ResponseSender 闭包
9:   调用 m_file_request_cb(msg, client_ip, sender)
10:  回调内部通过 sender() 发送 FILE_RESPONSE
11: else if type = DEVICE_HELLO then
12:   调用 m_device_hello_cb ▷ 注册设备 + ECDH
13:   if 本机有公钥 then 发送 DEVICE_HELLO_ACK
14: else if type = TEXT_MESSAGE then
15:   调用 m_control_msg_cb(msg, client_ip)
16:   发送 TEXT_ACK 响应
17: else if type = TRANSFER_* then
18:   调用 m_control_msg_cb(msg, client_ip)
19:   发送 CONTROL_ACK 响应
20: end if
21: close(client_sock)
```

3.7.2 信令通信时序

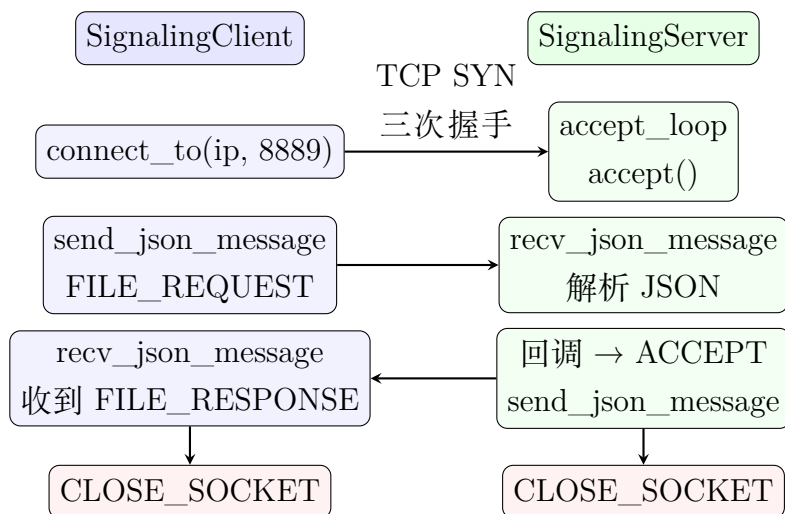


图 4: 信令通道短连接请求-响应流程 (以 FILE_REQUEST 为例)

信令通道的典型通信流程 (以发送 TEXT_MESSAGE 为例):

1. **客户端**: 调用 `SignalingClient::send_request(target_ip, 8889, request, response)`
2. `connect_to()`: 非阻塞 TCP 连接 + `select()` 超时等待 (默认 5s), 成功后恢复阻塞模式
3. `send_json_message()`: 发送 [4B 长度][JSON] 格式消息 (若 PeerKey 存在则自动 AES-256-GCM 加密)
4. **服务端**: `handle_client()` 接收消息, 根据 `type` 字段分发:
 - DEVICE_HELLO: 触发设备注册回调 + ECDH 密钥交换, 回复 DEVICE_HELLO_ACK
 - TEXT_MESSAGE: 触发回调缓存消息, 回复 TEXT_ACK
 - FILE_REQUEST: 触发回调 (ResponseSender 闭包), 回调内部调用 ACCEPT/REJECT 回复
 - TRANSFER_*: 触发控制回调, 回复 CONTROL_ACK
5. **双方**: 关闭连接 (`CLOSE_SOCKET`)

3.7.3 非阻塞连接与超时

`connect_to()` 实现细节:


```

1 SOCKET_FD SignalingClient::connect_to(target_ip, target_port, timeout_ms) {
2     SOCKET_FD sock = socket(AF_INET, SOCK_STREAM, 0);
3     NetworkUtils::set_nonblocking(sock); // 1. 设置非阻塞
4
5     // 2. 发起连接 (非阻塞模式立即返回, errno=EINPROGRESS)
6     int ret = connect(sock, ...);
7     if (ret < 0 && !NetworkUtils::is_would_block(err))
8         return INVALID_SOCKET_FD; // 真正的错误
9
10    // 3. select() 等待套接字可写 (连接完成)
11    fd_set write_fds; FD_ZERO(&write_fds); FD_SET(sock, &write_fds);
12    struct timeval tv; tv.tv_sec = timeout_ms / 1000;
13    int sel = select(sock+1, nullptr, &write_fds, nullptr, &tv);
14    if (sel <= 0) return INVALID_SOCKET_FD; // 超时或错误
15
16    // 4. 检查 SO_ERROR 确认连接成功 (select 可写不代表连接成功)
17    int so_error; socklen_t len = sizeof(so_error);
18    getsockopt(sock, SOL_SOCKET, SO_ERROR, &so_error, &len);
19    if (so_error != 0) return INVALID_SOCKET_FD; // 连接失败
20
21    NetworkUtils::set_blocking(sock); // 5. 恢复阻塞模式
22    return sock;
23 }

```

Listing 10: 非阻塞 connect + select 超时 (signaling_client.cpp)

3.8 TCP 文件传输流程

3.8.1 传输数据通道概述

文件传输数据通道（端口 8890）独立于信令通道，采用**长连接模式**（连接持续整个传输周期）。传输前通过信令通道完成握手（FILE_REQUEST → FILE_RESPONSE → ACCEPT），然后发送方连接接收方的传输端口开始数据传输。

与信令通道的关键区别：

表 3: 信令通道 vs 数据通道

特性	信令通道 (8889)	数据通道 (8890)
连接模式	TCP 短连接	TCP 长连接
消息格式	纯 JSON (仅长度前缀)	1 字节标记 + 载荷
标记类型	—	'J' JSON / 'C' Chunk / 'E' 加密 JSON / 'D' 加密 Chunk
并发连接	每请求一个	每线程一个 (多线程并行)
加密	JSON 层 AES-256-GCM	标记分发层 AES-256-GCM

3.8.2 数据传输时序 (多线程模式)

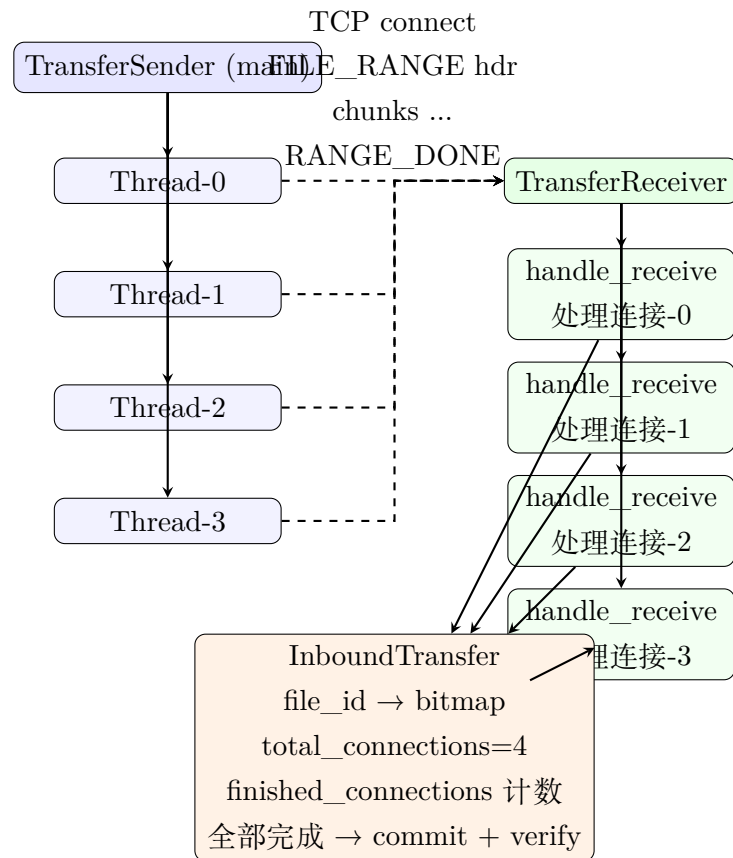


图 5: 多线程并行传输时序图 (4 线程, 1600 chunks, 每个线程 400 chunks)

3.8.3 Chunk 分片二进制协议

每个文件分片（Chunk）采用 16 字节定长头部 + 变长载荷的二进制格式，所有多字节整数均为大端序（网络字节序）：

表 4: Chunk 二进制格式

偏移	大小	字段
0	4 字节	chunk_index（分片序号）
4	4 字节	total_chunks（总分片数）
8	4 字节	data_size（载荷字节数）
12	4 字节	file_id_len（file_id 长度, 通常 36）
16	file_id_len	file_id（UUID 字符串）
16+L	data_size	文件数据（最大 64KB）

```
1 struct Chunk {
2     uint32_t chunk_index;
3     uint32_t total_chunks;
4     uint32_t data_size;
5     std::string file_id;
6     std::vector<uint8_t> data;
7
8     std::vector<uint8_t> serialize() {
9         size_t total = 16 + file_id.size() + data.size();
10        std::vector<uint8_t> buf(total);
11        write_u32_be(&buf[0], chunk_index);
12        write_u32_be(&buf[4], total_chunks);
13        write_u32_be(&buf[8], data_size);
14        write_u32_be(&buf[12], static_cast<uint32_t>(file_id.size()));
15        memcpy(&buf[16], file_id.data(), file_id.size());
16        memcpy(&buf[16 + file_id.size()], data.data(), data.size());
17        return buf;
18    }
19 };
```

Listing 11: Chunk 序列化 (chunk.h)

3.8.4 接收端多连接并发写入

接收端面临”N 条 TCP 连接同时向同一文件写入”的并发问题。解决方案：利用 POSIX `pwrite()` 的原子性。

```
1 bool FileChunkIO::write_chunk(const Chunk& chunk) {
2     // 计算分片在文件中的绝对偏移量
3     off_t offset = static_cast<off_t>(chunk.chunk_index) * m_chunk_size;
4
5     // pwrite() 是原子操作：直接在 offset 处写入，不修改文件指针
6     // 即使多个线程并发调用 pwrite(fd, ..., different_offsets),
7     // 内核保证每个写入是不重叠且原子的（不同偏移处无竞争）
8     ssize_t written = pwrite(m_fd, chunk.data.data(), chunk.data_size,
9                             offset);
10
11     // 标记该分片已接收 (bitmap 需要互斥锁保护)
12     {
13         std::lock_guard<std::mutex> lock(m_bitmap_mutex);
14         m_received_bitmap[chunk.chunk_index] = true;
15     }
16     return written == static_cast<ssize_t>(chunk.data_size);
17 }
```

Listing 12: FileChunkIO 并发安全写入 (file_io.cpp)

完成判定：InboundTransfer 跟踪每条连接的完成状态，当 `finished_connections == total_connections` 且 `received_count == total_chunks` 时，调用 `rename(temp_path, final_path)` 原子提交文件，并触发 `zlib` 解压和校验。

四、项目结果

4.1 测试环境

表 5: 测试环境配置

项目	发送端	接收端
主机名	OMEN	hhx
IP 地址	10.162.44.132	10.162.181.2
操作系统	Linux	Ubuntu Linux
编译环境	GCC, C++17	GCC, C++17
加密库	OpenSSL 3.x	OpenSSL 3.x
网络 RTT	—	186.8 ms

4.2 设备发现与信令测试

手动添加 IP 后，Probe 线程每 5 秒发送 DEVICE_HELLO 探活：

- 本地 10.162.44.132 POST /api/peers/add 添加 10.162.181.2
- 远程 10.162.181.2 POST /api/peers/add 添加 10.162.44.132
- ~15 秒后双向 DEVICE_HELLO 握手完成，ECDH 密钥协商成功

TEXT_MESSAGE 信令延迟测试（10 次采样）：

表 6: TEXT_MESSAGE 延迟统计 (ms)

指标	平均值	最小值	最大值
本地 → 远程	446.6	186.3	1033.3
远程 → 本地	1048.7	—	—

消息延迟 = TCP 连接建立 + JSON 序列化/反序列化 + 业务处理 + AES-256-GCM 加解密。平均延迟 $446.6\text{ms} \approx 2.4 \times \text{RTT}$ ，考虑到 TCP 三次握手（ 1.5 RTT ）+ 请求-响应（ 0.5 RTT ）+ 处理开销。

4.3 文件传输性能测试

4.3.1 吞吐量对比

表 7: 跨设备文件传输吞吐量对比

文件大小	线程数	吞吐量 (Mbps)	耗时 (s)	验证
10 MB	1	45.0	2.5	✓
10 MB	4	52.7	2.2	✓
100 MB	1	72.3	17.6	✓
100 MB	4	65.9	18.9	✓
500 MB	1	67.4	93.5	✓
500 MB	4	63.0	98.0	✓

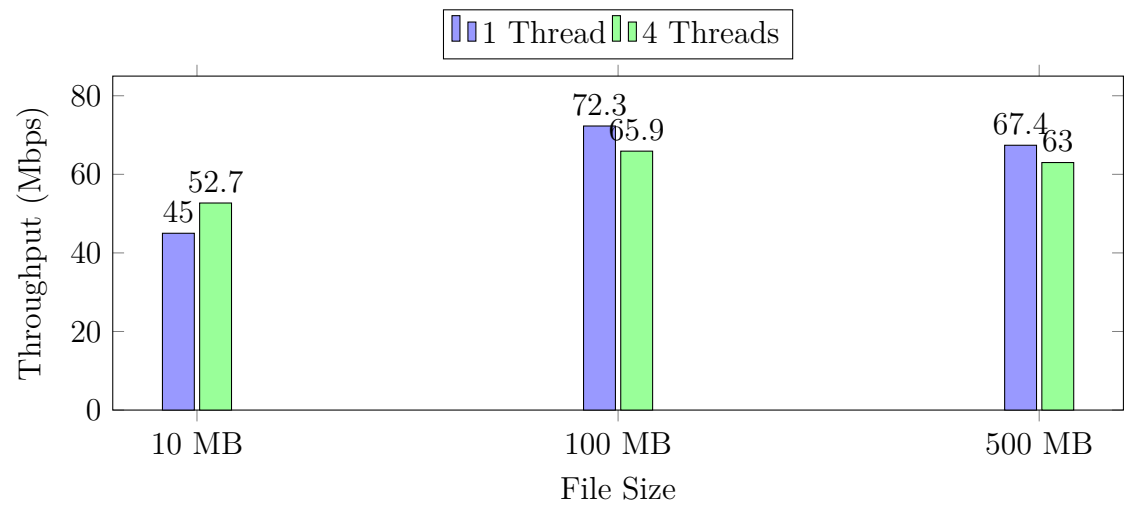


图 6: 不同文件大小与线程数的吞吐量对比

4.3.2 每线程延迟分解

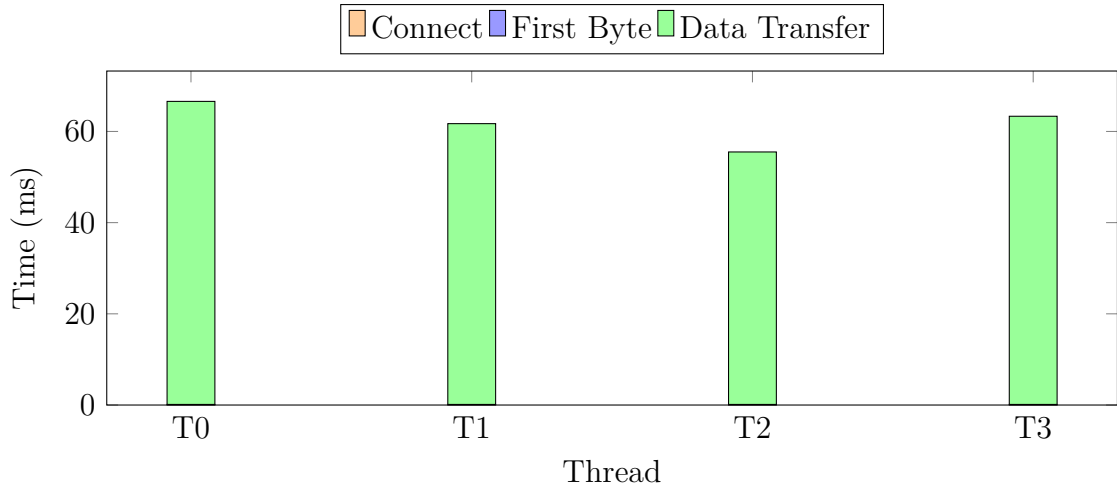


图 7: 500MB 文件 4 线程传输的延迟分解

4.4 结果分析

4.4.1 单线程已接近带宽上限

在 186.8ms RTT 的链路上, 单线程吞吐量稳定在 ~70 Mbps。假设链路物理带宽为 100 Mbps, 则带宽利用率为:

$$\eta = \frac{67.4 \text{ Mbps}}{\text{链路带宽}} \times 100\% \quad (13)$$

BBR 算法有效地探测并维持了瓶颈链路的可用带宽, 避免了传统的 CUBIC 算法在高 RTT 场景下需要长时间慢启动填满窗口的问题。

4.4.2 多线程加速效果分析

多线程并未带来显著的吞吐量提升, 甚至微降 (4 线程 500MB 从 67.4 降至 63.0 Mbps)。原因分析:

1. **链路带宽瓶颈**: 物理链路已饱和, 增加 TCP 连接只会引入更多竞争和上下文切换开销
2. **公平性**: TCP 拥塞控制保证多流公平共享瓶颈链路, N 条连接各分得 $\sim 1/N$ 带宽
3. **连接开销**: 每条连接的 TCP 握手 ($1.5 \text{ RTT} \approx 280\text{ms}$) 和 TLS-like 加密协商成为额外延迟
4. **适用场景**: 多线程在**高 BDP 网络** (如 1Gbps 链路 + 高 RTT) 中效果显著——此时单连接 TCP 窗口无法填满带宽, 多连接可聚合窗口

$$\text{理论加速比} = \min \left(N, \frac{B_{\text{link}}}{B_{\text{single}}} \right) \quad (14)$$

当 $B_{\text{single}} \approx B_{\text{link}}$ 时（单连接已饱和），加速比 ≈ 1 。

4.4.3 BBR vs 传统算法对比

BBR 在本测试中的核心贡献不是更高的峰值吞吐，而是**稳定的速率和平滑的延迟曲线**。在高 RTT (186ms) 下：

- CUBIC 依赖丢包作为拥塞信号，在无线/长肥网络中会因非拥塞丢包而大幅降低速率
- BBR 基于带宽和 RTT 的模型估计，不受随机丢包影响，10 MB $\rightarrow$ 100 MB 吞吐从 45 升至 72 Mbps（大文件窗口充分打开后稳定）

4.5 加密开销分析

AES-256-GCM 每 chunk (64KB) 引入 28 字节开销：

- IV: 12 字节（随机生成，每个消息唯一）
- GCM Tag: 16 字节（GHASH 认证码）
- 总开销: $\frac{28}{65536+28} \approx 0.043\%$ ，可忽略不计

CPU 加密开销：AES-NI 硬件指令加速下，GCM 加解密吞吐可达 GB/s 级别，对 ~ 70 Mbps 的链路不会成为瓶颈。

4.6 协议开销

表 8: 协议开销汇总

项目	开销 (字节)	占比 (64KB chunk)
Chunk 头部	16 + 36 (file_id)	0.08%
GCM IV + Tag	28	0.04%
JSON 消息头	~ 200	—
TCP/IP 头	40	0.06%

五、总结

本实验完成了一个完整的安全 P2P 文件传输系统，并通过跨网络实际测试验证了性能和安全性。主要成果包括：

1. **ECDH 密钥交换**：实现了基于 secp256r1 椭圆曲线的双向密钥协商，本地私钥 + 远程公钥计算 256 位共享密钥，直接作为 AES-256 加密密钥使用。
2. **AES-256-GCM 认证加密**：所有信令和数据通道消息均经过认证加密保护，12 字节随机 IV + 16 字节 GCM Tag 提供机密性和完整性，开销仅 0.04%。
3. **BBR 拥塞控制**：在每个 TCP 发送套接字上启用 BBR 算法，在高 RTT（186ms）链路上实现稳定吞吐，不受随机丢包影响。
4. **多线程并行传输**：将文件按 chunk 均匀分给 N 个线程，通过独立 TCP 连接并行发送，利用 `pwrite()` POSIX 原子写实现并发磁盘写入。
5. **跨网络验证**：在 10.162.44.132 与 10.162.181.2 之间完成完整的功能和性能测试，500MB 文件单线程吞吐 67.4 Mbps，所有传输文件 MD5 校验通过。

改进方向：

- 在更高带宽（1Gbps+）的链路上测试多线程加速效果，验证 BBR 在大 BDP 场景下的性能优势
- 引入 HKDF 密钥派生函数替代直接使用 ECDH 原始字节作为 AES 密钥，提高密钥安全性
- 增加断点续传功能的实际集成
- 支持 IPv6 和 NAT 穿透（STUN/TURN）